

Explicación Técnica y Algorítmica del Proyecto Financiero

Este documento detalla de manera exhaustiva el funcionamiento del código fuente por módulos, y profundiza en el análisis asintótico y mecánico de los algoritmos de ordenamiento y análisis implementados en el proyecto.

1. Desglose Técnico del Código (¿Qué hace cada parte?)

El proyecto está diseñado bajo una arquitectura modular estricta, dividiendo la responsabilidad de extracción de datos, análisis matemático y presentación visual en diferentes carpetas.

A. Punto de Entrada y Presentación (`main.py` y `app.py`)

- **`main.py`**: Es el orquestador inicial. Su única función es preparar el entorno local y lanzar el servidor web de Streamlit apuntando al archivo principal.
- **`app.py`**: Es el núcleo de la Interfaz de Usuario (UI). Utiliza Streamlit para maquetar el Dashboard financiero. Aquí se configuran las barras laterales (sidebars), las pestañas de navegación (tabs) y se invocan las funciones de las demás carpetas para renderizar los datos procesados en pantalla.

B. Módulo ETL (Extracción, Transformación y Carga) - Carpeta `etl/`

Este módulo es el responsable de conseguir y limpiar la materia prima (los datos financieros) sin usar librerías de alto nivel para el cruce de datos.

- **`extractor.py`**: Se conecta a la API de Yahoo Finance usando peticiones HTTP puras (`requests`). Descarga los precios históricos de las acciones (Tickers) y los guarda en formato crudo (archivos JSON en `data/raw/`).
- **`transformer.py`**: Aplica un proceso de limpieza. Los mercados financieros tienen días festivos donde no hay datos. Este archivo usa el algoritmo **LOCF (Last Observation Carried Forward)** para rellenar los datos vacíos con el último precio de cierre válido, asegurando que las series de tiempo sean continuas y sin "huecos" (guardando en `data/clean/`).
- **`loader.py`**: Une todos los archivos limpios individuales en un único gran repositorio consolidado (`master_dataset.csv`). Lo hace mediante diccionarios y algoritmos iterativos (Outer Join manual) para sincronizar las fechas exactas de todos los activos en una misma matriz estructurada.

C. Módulo Analítico - Carpeta `analysis/`

Aquí reside el motor matemático y financiero del proyecto.

- **`risk.py`**: Calcula los retornos logarítmicos diarios iterando sobre los precios de cierre ($\ln(P_t/P_{t-1})$). Con esto, halla la Desviación Estándar (Volatilidad) y clasifica automáticamente a los activos en perfiles de riesgo: Conservador, Moderado o Agresivo.

- **patterns.py**: Utiliza la técnica de **Sliding Window (Ventana Deslizante)**. Recorre el arreglo de datos en bloques (ej. de 3 en 3 días) para detectar patrones técnicos de Trading, como velas alcistas consecutivas o rupturas de tendencia.
- **similarity.py**: Implementa algoritmos matemáticos $O(N)$ y $O(N^2)$ para medir qué tan parecidas son dos acciones en su comportamiento. Incluye Distancia Euclidiana, Correlación de Pearson, Similitud del Coseno y **Dynamic Time Warping (DTW)** (este último usa programación dinámica matricial para alinear series de tiempo desfasadas).
- **ml.py**: Ejecuta simulaciones puras (Monte Carlo) usando iteraciones de fuerza bruta con caminatas aleatorias (`random.Gauss`) para simular miles de escenarios futuros del precio de una acción y aplicar teoría de portafolios de Markowitz.

D. Módulo de Renderizado - Carpeta **visualization/**

- **charts.py**: Recibe las matrices y arreglos matemáticos calculados en `analysis/` y los transforma en objetos visuales vectoriales usando `Plotly` (Gráficos de velas, mapas de calor, fronteras eficientes).

□ 2. Apartado Fuerte: Algoritmos de Ordenamiento (**sorting/algorithms.py**)

Para demostrar el manejo avanzado de estructuras de datos, el proyecto no utiliza las funciones de ordenamiento por defecto de Python (`.sort()`). En su lugar, se desarrollaron **12 algoritmos clásicos y avanzados desde cero**.

El criterio de ordenamiento unificado es: **Ordenar por Fecha Ascendente, y en caso de empate, por Precio de Cierre (Close) Ascendente**.

Algoritmos Básicos (Fuerza Bruta)

[!CAUTION] Estos algoritmos iteran múltiples veces sobre la matriz de datos, siendo ineficientes para el Big Data pero excelentes por su bajo consumo de memoria (In-place).

1. Selection Sort (Selección)

- **Mecánica**: Recorre la lista buscando el elemento más pequeño y lo intercambia con la primera posición no ordenada.
- **Complejidad**: $O(N^2)$ temporal, $O(1)$ espacial. Siempre hace el mismo número de comprobaciones, sin importar si la lista ya está ordenada.

```
def selection_sort(arr):
    start_time = time.time()
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if _es_menor(arr[j], arr[min_idx]):
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr, time.time() - start_time
```

2. Gnome Sort (El Gnomo de Jardín)

- **Mecánica:** Un gnomo mira la maceta actual y la anterior. Si están en el orden correcto, da un paso adelante; si no, las intercambia y da un paso atrás.
- **Complejidad:** $O(N^2)$ en el peor caso. Muy eficiente ($O(N)$) si la lista ya está casi ordenada.

```
def gnome_sort(arr):
    start_time = time.time()
    index = 0
    n = len(arr)
    while index < n:
        if index == 0 or not _es_menor(arr[index], arr[index - 1]):
            index += 1
        else:
            arr[index], arr[index - 1] = arr[index - 1], arr[index]
            index -= 1
    return arr, time.time() - start_time
```

Algoritmos Intermedios e Inserción

[!NOTE] Mejoran las comparaciones aprovechando espacios ya organizados en la memoria.

3. Binary Insertion Sort

- **Mecánica:** Evolución del Insertion Sort. En lugar de buscar linealmente dónde insertar un elemento, usa **Búsqueda Binaria** (dividiendo el sub-arreglo ordenado a la mitad) para encontrar la posición en $O(\log N)$ comparaciones.
- **Complejidad:** $O(N^2)$ por los desplazamientos de memoria, pero las comparaciones se reducen drásticamente a $O(N \log N)$.

```
def binary_insertion_sort(arr):
    start_time = time.time()
    for i in range(1, len(arr)):
        val = arr[i]
        left, right = 0, i - 1
        while left <= right:
            mid = (left + right) // 2
            if _es_menor(val, arr[mid]):
                right = mid - 1
            else:
                left = mid + 1
        for j in range(i, left, -1):
            arr[j] = arr[j - 1]
        arr[left] = val
    return arr, time.time() - start_time
```

4. Comb Sort (Peine)

- **Mecánica:** Una mejora masiva del Bubble Sort. Elimina las "tortugas" (valores pequeños al final de la lista) comparando elementos separados por un "gap" (salto) amplio que se reduce iterativamente multiplicándolo por un factor de contracción (ej. 1.3).
- **Complejidad:** Promedio esperado $O(N \log N)$, Espacial $O(1)$.

```
def comb_sort(arr):
    start_time = time.time()
    n = len(arr)
    gap = n
    shrink = 1.3
    swapped = True
    while gap > 1 or swapped:
        gap = int(float(gap) / shrink)
        if gap < 1: gap = 1
        swapped = False
        for i in range(0, n - gap):
            if _es_mayor(arr[i], arr[i + gap]):
                arr[i], arr[i + gap] = arr[i + gap], arr[i]
                swapped = True
    return arr, time.time() - start_time
```

Algoritmos Divide y Vencerás (Divide and Conquer)

[!TIP] Estos son los caballos de batalla de la ciencia computacional moderna, utilizados para procesar los años de historial financiero en fracciones de segundo.

5. Quick Sort (Pivoteo recursivo)

- **Mecánica:** Escoge un elemento (pivote) del centro de la matriz. Divide los datos en tres listas: menores al pivote, iguales, y mayores. Se llama a sí mismo recursivamente en las sublistas.
- **Complejidad:** $O(N \log N)$ promedio.

```
def quick_sort(arr):
    start_time = time.time()
    def _qs(a):
        if len(a) <= 1:
            return a
        pivot = a[len(a) // 2]
        left = []
        middle = []
        right = []
        for x in a:
            if _es_menor(x, pivot):
                left.append(x)
            elif _es_mayor(x, pivot):
                right.append(x)
            else:
                middle.append(x)
        return _qs(left) + middle + _qs(right)

    result = _qs(arr)
    return result, time.time() - start_time
```

6. Heap Sort (Montículos Binarios)

- **Mecánica:** Convierte la lista tabular en un Árbol Binario Completo (Max-Heap) mediante saltos de índices ($2*i + 1$, $2*i + 2$). El valor máximo siempre sube a la

raíz (0) y luego se extrae y se coloca al final de la lista.

- **Complejidad:** $O(N \log N)$ garantizado incluso en el peor caso. Altamente eficiente en memoria $O(1)$.

```
def heap_sort(arr):
    start_time = time.time()
    n = len(arr)

    def heapify(n_size, i):
        largest = i
        l = 2 * i + 1
        r = 2 * i + 2
        if l < n_size and _es_mayor(arr[l], arr[largest]):
            largest = l
        if r < n_size and _es_mayor(arr[r], arr[largest]):
            largest = r
        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(n_size, largest)

    for i in range(n // 2 - 1, -1, -1):
        heapify(n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(i, 0)
    return arr, time.time() - start_time
```

7. Tree Sort (Árbol Binario de Búsqueda)

- **Mecánica:** Inserta cada registro financiero como un Nodo de un árbol (izquierdo si es menor fecha, derecho si es mayor). Una vez construido, recorre el árbol en orden (*In-Order Traversal*) para extraer los datos ordenados.
- **Complejidad:** $O(N \log N)$ temporal, pero consume $O(N)$ en espacio.

```
def tree_sort(arr):
    start_time = time.time()
    if not arr: return arr, 0.0

    def transversa_en_orden(nodo, salida):
        if nodo is not None:
            transversa_en_orden(nodo.izquierdo, salida)
            salida.append(nodo.dato)
            transversa_en_orden(nodo.derecho, salida)

    raiz = Nodo(arr[0])
    longitud = len(arr)
    for i in range(1, longitud):
        current = raiz
        elem = arr[i]
        while True:
            if _es_menor(elem, current.dato):
                if current.izquierdo is None:
                    current.izquierdo = Nodo(elem)
                    break
```

```

        else:
            current = current.izquierdo
    else:
        if current.derecho is None:
            current.derecho = Nodo(elem)
            break
        else:
            current = current.derecho

res = []
transversa_en_orden(raiz, res)
return res, time.time() - start_time

```

8. Tim Sort (Customizado)

- **Mecánica:** Un híbrido que divide la data en bloques pequeños (*Runs* de 32 elementos), los ordena con Insertion Sort y luego los fusiona usando el algoritmo Merge Sort.
- **Complejidad:** $O(N \log N)$ garantizado y altamente estable.

```

def tim_sort(arr):
    start_time = time.time()
    MIN_MERGE = 32
    n = len(arr)

    def calc_min_run(n):
        r = 0
        while n >= MIN_MERGE:
            r |= n & 1
            n >>= 1
        return n + r

    min_run = calc_min_run(n)

    def __insercion_ts(izq, der):
        for i in range(izq + 1, der + 1):
            j = i
            while j > izq and _es_menor(arr[j], arr[j - 1]):
                arr[j], arr[j - 1] = arr[j - 1], arr[j]
                j -= 1

    def __mezcla_ts(l, m, r):
        len1, len2 = m - l + 1, r - m
        left_p, right_p = [], []
        for i in range(0, len1): left_p.append(arr[l + i])
        for i in range(0, len2): right_p.append(arr[m + 1 + i])

        i, j, k = 0, 0, l
        while i < len1 and j < len2:
            if not _es_mayor(left_p[i], right_p[j]):
                arr[k] = left_p[i]
                i += 1
            else:
                arr[k] = right_p[j]

```

```

        j += 1
        k += 1
    while i < len1:
        arr[k] = left_p[i]
        k += 1
        i += 1
    while j < len2:
        arr[k] = right_p[j]
        k += 1
        j += 1

for i in range(0, n, min_run):
    __insercion_ts(i, min((i + MIN_MERGE - 1), (n - 1)))

size = min_run
while size < n:
    for left in range(0, n, 2 * size):
        mid = min((left + size - 1), (n - 1))
        right = min((left + 2 * size - 1), (n - 1))
        if mid < right:
            __mezcla_ts(left, mid, right)
    size *= 2

return arr, time.time() - start_time

```

Algoritmos No Comparativos (Basados en Distribución)

[!IMPORTANT] A diferencia de los anteriores, estos algoritmos NO usan los operadores matemáticos de menor o mayor (<, >). Ordenan agrupando elementos en "Cajas" usando características de los datos.

9. Pigeonhole Sort (Casillas de Palomas)

- **Mecánica:** Crea un arreglo auxiliar con un "agujero" para cada posible fecha existente en el rango. Coloca cada activo directamente en su agujero.
- **Complejidad:** $O(N + \text{Rango})$. Muy veloz, pero terrible en memoria si hay huecos vacíos.

```

def pigeonhole_sort(arr):
    start_time = time.time()
    if not arr: return arr, 0.0

    def fecha_int(d): return int(d.replace("-", "").[:6])
    mi = fecha_int(arr[0]['fecha'])
    ma = mi
    for x in arr:
        v = fecha_int(x['fecha'])
        if v < mi: mi = v
        if v > ma: ma = v

    size = ma - mi + 1
    holes = [[] for _ in range(size)]
    for x in arr: holes[fecha_int(x['fecha']) - mi].append(x)

```

```

out = []
for hole in holes:
    if hole:
        hole_s, _ = quick_sort(hole)
        for e in hole_s: out.append(e)

return out, time.time() - start_time

```

10. Bucket Sort (Cubetas Anuales)

- **Mecánica:** Divide los datos creando una lista/cubeta por cada **Año** financiero. Luego aplica Quick Sort dentro de cada cubeta y finalmente une todos los años.
- **Complejidad:** $O(N + K)$. Excelente para dispersar cargas computacionales.

```

def bucket_sort(arr):
    start_time = time.time()
    if not arr: return arr, 0.0

    cubetas = {}
    for elemento in arr:
        ano = int(elemento['fecha'][:4])
        if ano not in cubetas: cubetas[ano] = []
        cubetas[ano].append(elemento)

    claves = []
    for k in cubetas: claves.append(k)
    for i in range(len(claves)):
        for j in range(i + 1, len(claves)):
            if claves[j] < claves[i]: claves[i], claves[j] = claves[j],

    out = []
    for k in claves:
        c, _ = quick_sort(cubetas[k])
        for x in c: out.append(x)

    return out, time.time() - start_time

```

11. Radix Sort (Ordenamiento por Raíz / Dígitos)

- **Mecánica:** Convierte las fechas a números enteros. Ordena los datos procesando dígito por dígito de derecha a izquierda, usando Counting Sort.
- **Complejidad:** $O(d \times (N + b))$ donde d es la cantidad de dígitos.

```

def radix_sort(arr):
    start_time = time.time()
    if not arr: return arr, 0.0

    def dt_val(d): return int(d['fecha'].replace("-", ""))
    val_maximo = max(dt_val(x) for x in arr)

    def counting_sort_radix(arreglo_d, exp):
        longitud = len(arreglo_d)
        salida = [0] * longitud

```

```

    conteo = [0] * 10
    for i in range(longitud):
        indice = (dt_val(arreglo_d[i]) // exp) % 10
        conteo[indice] += 1
    for i in range(1, 10): conteo[i] += conteo[i - 1]
    for i in range(longitud - 1, -1, -1):
        indice = (dt_val(arreglo_d[i]) // exp) % 10
        salida[conteo[indice] - 1] = arreglo_d[i]
        conteo[indice] -= 1
    return salida

arreglo_work = arr[:]
arreglo_work = counting_sort_radix(arreglo_work, 1)
exponente = 10
while val_maximo // exponente > 0:
    arreglo_work = counting_sort_radix(arreglo_work, exponente)
    exponente *= 10

out, _ = tim_sort(arreglo_work)
return out, time.time() - start_time

```

Algoritmo de Hardware / Paralelismo

12. Bitonic Sort

- **Mecánica:** Diseñado originalmente para tarjetas gráficas (GPU) y circuitos paralelos. Crea secuencias que suben y bajan repetidamente (Bitónicas) y las fusiona. Requiere tamaño potencia de 2 (2^x).
- **Complejidad:** $O(N \log^2 N)$ iterativo.

```

def bitonic_sort(arr):
    start_time = time.time()
    longitud = len(arr)
    if longitud <= 1: return arr, time.time() - start_time

    potencia = 1
    while potencia < longitud: potencia *= 2

    val_relleno = {'fecha': '9999-99-99', 'close': float('inf'), 'volumen': 0}
    longitud_relleno = potencia - longitud

    arreglo_c = []
    for x in arr: arreglo_c.append(x)
    for _ in range(longitud_relleno): arreglo_c.append(val_relleno)

    def c_swap(a, i, j, d_):
        if (d_ == 1 and _es_mayor(a[i], a[j])) or (d_ == 0 and _es_menor(a[i], a[j])):
            a[i], a[j] = a[j], a[i]

    def _mergest(a, l_idx, count, d_):
        if count > 1:
            k = count // 2
            for i in range(l_idx, l_idx + k):

```

```

        c_swap(a, i, i + k, d_)
        _mergest(a, l_idx, k, d_)
        _mergest(a, l_idx + k, k, d_)

def _bitonic_st(a, l_idx, count, d_):
    if count > 1:
        k = count // 2
        _bitonic_st(a, l_idx, k, 1)
        _bitonic_st(a, l_idx + k, k, 0)
        _mergest(a, l_idx, count, d_)

_bitonic_st(arreglo_c, 0, potencia, 1)

out = []
for i in range(longitud):
    if arreglo_c[i]['fecha'] != '9999-99-99':
        out.append(arreglo_c[i])

return out, time.time() - start_time

```

3. Análisis de Algoritmos de Similitud (Requerimiento 2)

El Requerimiento 2 del proyecto exige la implementación manual de algoritmos que midan qué tan parecidas son dos series de tiempo (acciones o ETFs). No se utilizaron librerías de alto nivel como `scipy.spatial.distance` o `sklearn.metrics`. Todo se iteró desde cero.

1. Distancia Euclidiana

- **Explicación Matemática:** Calcula la longitud del segmento de recta que conecta dos puntos en un espacio N-dimensional. La fórmula es $d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$.
- **Mecánica Algorítmica:** Itera simultáneamente sobre ambas series de tiempo, calcula la diferencia de los precios en el día i , la eleva al cuadrado y la suma a un acumulador. Al finalizar, saca la raíz cuadrada.
- **Complejidad Computacional:** Temporal $O(N)$ porque hace un único recorrido lineal. Espacial $O(1)$.

```

def euclidean_distance(series1, series2):
    n = len(series1)
    if n == 0: return 0.0, "O(N) "

    sum_sq = 0.0
    for i in range(n):
        sum_sq += (series1[i] - series2[i]) ** 2

    return sum_sq ** 0.5, "O(N) "

```

2. Correlación de Pearson

- **Explicación Matemática:** Mide la dependencia lineal entre dos variables. Su fórmula es $\frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$. Retorna un valor entre -1 y 1.
- **Mecánica Algorítmica:** En una sola iteración $O(N)$, acumula la suma de X, suma de Y, producto XY y los cuadrados X^2 y Y^2 . Luego aplica la fórmula algebraica para hallar el numerador y denominador.
- **Complejidad Computacional:** Temporal $O(N)$ y Espacial $O(1)$.

```
def pearson_correlation(series1, series2):
    n = len(series1)
    if n == 0: return 0.0, "O(N) "

    sum_x, sum_y, sum_xy, sum_x2, sum_y2 = 0.0, 0.0, 0.0, 0.0, 0.0

    for i in range(n):
        x = float(series1[i])
        y = float(series2[i])
        sum_x += x; sum_y += y
        sum_xy += x * y
        sum_x2 += x * x; sum_y2 += y * y

    numerator = n * sum_xy - sum_x * sum_y
    denominator = ((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2)) *
    if denominator == 0: return 0.0, "O(N) "
    return numerator / denominator, "O(N) "
```

3. Similitud del Coseno

- **Explicación Matemática:** Mide el coseno del ángulo entre dos vectores proyectados en un espacio multidimensional. Fórmula: $\frac{A \cdot B}{||A|| ||B||}$.
- **Mecánica Algorítmica:** Calcula el producto punto (dot product) en el numerador y las normas euclidianas (magnitud) de cada vector en el denominador.
- **Complejidad Computacional:** Temporal $O(N)$ y Espacial $O(1)$.

```
def cosine_similarity(series1, series2):
    n = len(series1)
    if n == 0: return 0.0, "O(N) "

    dot_product, norm_a_sq, norm_b_sq = 0.0, 0.0, 0.0

    for i in range(n):
        a = float(series1[i])
        b = float(series2[i])
        dot_product += a * b
        norm_a_sq += a * a
        norm_b_sq += b * b

    norm_a = norm_a_sq ** 0.5
    norm_b = norm_b_sq ** 0.5
    if norm_a == 0 or norm_b == 0: return 0.0, "O(N) "
    return dot_product / (norm_a * norm_b), "O(N) "
```

4. Dynamic Time Warping (DTW)

- **Explicación Matemática:** A diferencia de la Euclidiana (que compara el día 1 con el día 1), DTW permite comparar secuencias desfasadas en el tiempo, emparejando puntos similares aunque ocurran en diferentes momentos.
- **Mecánica Algorítmica:** Usa **Programación Dinámica**. Crea una matriz de tamaño $(N+1) \times (M+1)$ inicializada en infinito. Luego itera llenando cada celda con el costo absoluto entre los dos puntos más el costo mínimo acumulado de sus celdas vecinas anteriores (izquierda, arriba, diagonal).
- **Complejidad Computacional:** Temporal $O(N \times M)$ (cuadrática si ambas series son de tamaño N). Espacial $O(N \times M)$ por la matriz de cálculo.

```
def dtw_distance(s1, s2):
    n, m = len(s1), len(s2)
    dtw_matrix = np.full((n + 1, m + 1), fill_value=np.inf)
    dtw_matrix[0, 0] = 0

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = abs(s1[i - 1] - s2[j - 1])
            last_min = np.min([dtw_matrix[i - 1, j], dtw_matrix[i, j - 1], dtw_matrix[i - 1, j - 1]])
            dtw_matrix[i, j] = cost + last_min

    return dtw_matrix[n, m], "O(N*M) "
```